🏥 Multi-Agent Medical Assistant - Project Capabilities

📋 Overview

The **Multi-Agent Medical Assistant** is an AI-powered, multi-agentic system designed to assist with medical diagnosis, research, and patient interactions. It combines multiple specialized agents to provide comprehensive medical assistance through intelligent routing, advanced reasoning, and real-time data integration.

🎯 Core Capabilities

1. **Multi-Agent Orchestration & Intelligent Routing**

- **Dynamic Agent Routing**: Automatically routes queries to the most appropriate specialized agent based on query content and context
- **Available Agents**:
  - **Conversation Agent**: Handles general chat, greetings, and non-medical questions
  - **RAG Agent**: Retrieves specific medical knowledge from ingested medical literature
  - **Web Search Processor Agent**: Fetches latest medical research and current health information
  - **Brain Tumor Analysis Agent**: Analyzes brain MRI images for tumor detection and segmentation
  - **Chest X-Ray Analysis Agent**: Analyzes chest X-ray images for disease detection and COVID-19 diagnosis
  - **Skin Lesion Analysis Agent**: Classifies skin lesion images as benign or malignant
- **Confidence-Based Agent Handoff**: Automatically transfers from RAG to Web Search when confidence falls below threshold (0.40)
- **Context Preservation**: Maintains conversation history across agent transitions

---

2. **Advanced Retrieval-Augmented Generation (RAG) System**

Document Processing

- **Docling-Based Parsing**: Extracts text, tables, and images from medical PDFs with high accuracy
- **Semantic Chunking**: LLM-based intelligent chunking that respects document structure and boundaries
- **Multi-Format Embedding**: Embeds text, tables, and LLM-generated image summaries together

Knowledge Retrieval

- **Vector Database**: Uses Qdrant for scalable, efficient vector storage and retrieval
- **Hybrid Search**: Combines:
  - **Dense Semantic Search**: Vector-based similarity search on medical document embeddings
  - **Sparse Keyword Search**: BM25-based keyword matching for precise term matching
  - **Re-ranking**: HuggingFace Cross-Encoder (ms-marco-TinyBERT-L-6) re-ranks retrieved chunks for accuracy

Query Enhancement

- **Query Expansion**: Automatically expands queries with related medical domain terminology
- **Conversation Context**: Includes last 20 messages (10 Q&A pairs) in retrieval pipeline

Response Generation

- **Context-Aware Responses**: Generates responses grounded in retrieved medical literature
- **Source Attribution**: Provides links to reference documents and images used in responses
- **Confidence Scoring**: Calculates confidence in retrieved information for decision-making

Pre-Ingested Medical Knowledge

- **Brain Tumors**: Detection techniques, deep learning methods, MRI analysis
- **COVID-19/Pneumonia**: Chest X-ray findings, disease progression analysis
- **Diabetes Mellitus**: Clinical information and management
- **Skin Lesions**: Melanoma detection, classification techniques

---

3. **Medical Imaging Analysis**

Computer Vision Capabilities

The system processes medical images with specialized deep learning models:

Chest X-Ray Analysis

- **Purpose**: COVID-19 and pneumonia detection from chest X-ray images
- **Model**: PyTorch-based classification model
- **Outputs**: Disease classification, confidence scores, clinical insights
- **Use Case**: Rapid screening for respiratory diseases

Skin Lesion Analysis

- **Purpose**: Benign vs. malignant skin lesion classification and segmentation
- **Model**: PyTorch-based semantic segmentation
- **Outputs**: Lesion classification, segmentation visualization, malignancy risk assessment
- **Use Case**: Early melanoma detection, dermatological screening

Brain Tumor Detection (Framework Ready)

- **Purpose**: Brain MRI analysis for tumor detection and segmentation
- **Architecture**: Object detection model ready for integration
- **Expected Outputs**: Tumor localization, size estimation, segmentation masks
- **Use Case**: Oncology diagnosis support

Image Processing Features

- **Automatic Image Classification**: Detects if uploaded image is medical or non-medical
- **Image Type Detection**: Automatically identifies chest X-rays, skin lesions, brain MRIs
- **Visualization Outputs**: Generates segmentation plots and analysis visualizations
- **Batch Processing Ready**: Infrastructure for processing multiple images

---

4. **Voice Interaction Capabilities**

Speech-to-Text

- Converts spoken medical queries into text for processing
- Seamless audio input capture from web interface

Text-to-Speech

- **Provider**: Eleven Labs API integration
- **Functionality**: Converts medical responses into natural-sounding audio

- **Use Cases**: Accessibility for visually impaired users, multitasking capability
- **Customizable**: Support for different voice personalities and languages

Audio Processing

- **Format Support**: MP3, WAV audio conversion via pydub
- **Background Cleanup**: Automatic cleanup of old audio files every 5 minutes
- **Session Management**: Temporary storage with automatic cleanup

---

5. **Human-in-the-Loop Validation System**

Medical Professional Oversight

- **Workflow Interruption**: Pauses AI decisions when validation is needed
- **State Persistence**: Stores interrupted workflow state for resumption
- **Review Interface**: Provides medical professionals with structured validation UI

Validation Process

1. AI generates initial diagnosis/analysis
2. System flags response needing validation (for medical imaging)
3. Validation interface presented to medical reviewer
4. Professional approves, modifies, or rejects AI output
5. Workflow resumes with validated information

Use Cases

- Verifying medical image analysis before presenting to patients
- Reviewing AI-generated medical recommendations
- Quality assurance in diagnostic pipeline
- Building confidence in AI-assisted diagnosis

---

6. **Safety & Guardrails System**

Input Guardrails

- **Query Validation**: Filters inappropriate or harmful medical queries
- **Content Safety**: Prevents requests for dangerous medical procedures
- **Medical Relevance**: Ensures queries are medically appropriate

Output Guardrails

- **Response Safety**: Filters potentially harmful medical recommendations
- **Bias Prevention**: Removes biased or discriminatory medical advice

- **Hallucination Prevention**: Ensures responses are grounded in retrieved knowledge
- **Confidence Checks**: Prevents high-confidence claims without sufficient evidence

Guardrail Features

- **Customizable Thresholds**: Adjustable confidence levels for different medical contexts
- **Fallback Routing**: Routes to web search or human review when confidence is low
- **Audit Logging**: Tracks all guardrail triggers for quality monitoring

---

7. **User Interface & Accessibility**

Web-Based Chat Interface

- **Responsive Design**: HTML/CSS/JavaScript frontend
- **Conversation Management**: Display of full conversation history
- **Session Handling**: Persistent sessions via cookies and database

Input Methods

- **Text Input**: Standard text query entry
- **Image Upload**: Drag-and-drop or file selection for medical images
- **Voice Input**: Audio recording for speech-to-text
- **File Formats**: Supports PNG, JPG, JPEG medical images

Output Display

- **Formatted Responses**: Rich text formatting for medical information
- **Image Visualization**: Displays segmentation outputs and analysis results
- **Source Links**: Clickable links to reference documents and images
- **Audio Output**: Plays text-to-speech responses

---

8. **Backend Architecture & Scalability**

FastAPI Backend

- **Async Processing**: Non-blocking request handling for performance
- **RESTful Endpoints**: Well-defined API routes for all operations
- **Health Checks**: `/health` endpoint for monitoring and Docker health verification
- **Session Management**: Cookie-based session tracking

Endpoints

- `GET /` - Serves main chat interface
- `POST /chat` - Processes text queries through multi-agent system
- `POST /analyze-image` - Processes uploaded medical images
- `POST /synthesize-speech` - Generates text-to-speech audio
- `GET /health` - System health check
- `POST /validate` - Handles human validation feedback

Storage & Persistence

- **Uploads Directory**: Manages user-uploaded images and outputs
- **Data Storage**: Local filesystem storage for documents and parsed content
- **Session Storage**: Memory-based session management with checkpoints
- **Temporary Files**: Automatic cleanup of old audio and temporary files

---

9. **Configuration & Customization**

Model Selection

- **LLM Models**: Configurable language models (default: Ollama Gemma4)
- **Embedding Models**: Customizable embedding models for vector search
- **Vision Models**: Configurable computer vision models for image analysis
- **Re-ranking Models**: Swappable cross-encoder models for result re-ranking

Temperature & Behavior Tuning

- **Agent-Specific Temperature**: Different temperature values for different agents
    - Decision Agent: 0.1 (deterministic routing)
    - Conversation Agent: 0.7 (creative responses)
    - RAG Agent: 0.3 (balanced accuracy and relevance)
    - Web Search: 0.3 (focused responses)

Retrieval Parameters

- **Chunk Size**: Adjustable document chunking (default: 512 tokens)
- **Chunk Overlap**: Configurable overlap for context continuity (default: 50 tokens)
- **Top-K Results**: Adjustable number of retrieved documents (default: 5)
- **Re-rank Top-K**: Number of re-ranked results (default: 3)
- **Confidence Threshold**: Minimum confidence for RAG responses (default: 0.40)
- **Context Limit**: Maximum conversation history messages (default: 20)

---

10. **Data Pipeline & Processing**

Document Ingestion

- **Format Support**: PDF documents with complex formatting
- **Batch Processing**: `ingest_rag_data.py` script for bulk document ingestion
- **Parsing**: Docling-based parsing extracts:
  - Structured text content
  - Tables with preservation of relationships
  - Images with LLM-based summaries
  - Document metadata

Data Storage

- **Raw Documents**: Original PDF files stored in `data/raw/`
- **Parsed Content**: Extracted text, tables, and images in `data/parsed_docs/`
- **Vector Store**: Qdrant database for embeddings in `data/qdrant_db/`
- **Document Database**: Individual document chunks in `data/docs_db/`

Processing Pipeline

1. Raw documents ingested from `raw/` directory
2. Docling parser extracts content (text, tables, images)
3. Content stored in parsed format
4. LLM-based semantic chunking applied
5. Chunks embedded using configured embedding model
6. Embeddings stored in Qdrant vector database
7. BM25 indexes created for keyword search

---

11. **Deployment & Containerization**

Docker Support

- **Containerized Deployment**: Complete Dockerfile for production deployment
- **Environment Configuration**: .env file support for API keys and settings
- **Volume Mounting**: Data persistence through Docker volumes
- **Port Mapping**: Configurable port for FastAPI server

CI/CD Integration

- **GitHub Actions**: Automated testing and deployment workflows
- **Build Automation**: Automated Docker image building and pushing

- **Version Management**: Semantic versioning with release tags

---

12. **Integration Capabilities**

Third-Party Services

- **Eleven Labs API**: For high-quality text-to-speech synthesis
- **Tavily Search**: Real-time web search for current medical information
- **PubMed Search**: Access to medical research papers and publications
- **Qdrant Vector Database**: Cloud or local vector storage

LLM Integration

- **OpenAI**: GPT-4o models (if configured)
- **Azure OpenAI**: Enterprise deployment option
- **Ollama**: Local model running capability
- **LangChain**: LLM framework with unified interface

🔄 Typical Workflows

Workflow 1: Medical Knowledge Query

```
User Query → Router → RAG Agent → Vector Search + Re-ranking →
Response Generation → Source Attribution → Output to User
```

Workflow 2: Medical Image Analysis

```
Image Upload → Router → Image Classifier → Specific Medical Ag
Analysis Model → Human Validation (Optional) → Response with V
```

Workflow 3: Current Medical Research Query

```
User Query (Recent Topic) → Router → Web Search Agent →
Web Search + PubMed → Synthesis → Response with Links
```

Workflow 4: Low Confidence Handoff

```
RAG Query → Insufficient Confidence (< 0.40) → Handoff to Web
Current Data Retrieval → Response with Both Sources
```

📊 Supported Medical Domains

Pre-Ingested Knowledge

1. **Neuro-Oncology**: Brain tumor detection and analysis
2. **Pulmonology**: COVID-19 and pneumonia diagnosis from chest X-rays
3. **Dermatology**: Skin lesion analysis and melanoma detection
4. **Endocrinology**: Diabetes mellitus information

Expandable Domains

The system is designed to be extended with additional:

- Medical specialties through document ingestion
- Image analysis types through new CV models
- Data sources through web search integration

🛠️ Technical Stack Summary

Component	Technology
Backend Framework	FastAPI
Agent Orchestration	LangGraph + LangChain
Document Parsing	Docling
Vector Database	Qdrant
LLM	Ollama (Gemma4) / OpenAI / Azure
Embeddings	Ollama / Azure OpenAI
CV Models	PyTorch-based (Chest X-ray, Skin Lesion)
Speech	Eleven Labs API
Frontend	HTML5, CSS3, JavaScript (Vanilla)
Deployment	Docker
Search	Tavily + PubMed

🚀 Key Differentiators

✅ **Multi-Agentic Architecture**: Specialized agents for different medical tasks
✅ **Hybrid RAG System**: Combines semantic + keyword search with re-ranking
✅ **Human-in-the-Loop**: Medical professional validation integrated
✅ **Medical Imaging**: Computer vision for multiple modalities
✅ **Voice-Enabled**: Complete audio input/output capability
✅ **Safety First**: Built-in guardrails and confidence scoring
✅ **Production-Ready**: Containerized, scalable, modular design
✅ **Knowledge Expandable**: Easy document ingestion pipeline

🎓 Learning Value

This project demonstrates:

- **Multi-agent orchestration** with LangGraph for complex AI workflows
- **Advanced RAG** techniques: semantic chunking, hybrid search, re-ranking
- **Computer vision** integration for medical image analysis
- **Production architecture** with FastAPI, Docker, and modular design
- **Human-AI collaboration** through validation systems
- **Guardrails and safety** in medical AI applications
- **Voice interaction** for accessible medical assistants
- **End-to-end AI pipeline** from data ingestion to user interface

---

📝 Notes

- **Version**: v2.1+ (uses Docling for document parsing)
- **License**: Apache 2.0
- **Status**: Active development with contributions welcome
- **Python Version**: 3.11+

For detailed technical documentation, see the individual README files in the `agents/` directory.